TASK 2: PROJECT QUERIES

Project Title

Movie Recommendation System Using MySQL

SUJIT KUMAR
ROLL NO: 2410030523

Problem Statement

The Movie Recommendation System is designed to store and manage information related to movies, users, and their ratings.

The system helps in recommending movies to users based on genre preference, user ratings, popularity, and unseen movies using SQL queries.

This project demonstrates the use of SELECT queries, joins, aggregate functions, subqueries, and DML operations (INSERT, UPDATE, DELETE) in MySQL.

SQL QUERIES

```
MariaDB [(none)]> CREATE DATABASE movie_recommendation_system;
Query OK, 1 row affected (0.003 sec)

MariaDB [(none)]> show databases;
+----------------------------+
| Database                   |
+----------------------------+
| 2cse4                      |
| information_schema         |
| movie_recommendation_system |
| mysql                      |
| performance_schema         |
| phpmyadmin                 |
| test                       |
+----------------------------+
7 rows in set (0.002 sec)
```

```
MariaDB [movie_recommendation_system]> CREATE TABLE users (
    ->     user_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     username VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.026 sec)

MariaDB [movie_recommendation_system]> show tables
    -> ;
+----------------------------------------+
| Tables_in_movie_recommendation_system |
+----------------------------------------+
| movies                                 |
| users                                  |
+----------------------------------------+
2 rows in set (0.001 sec)
```

```
MariaDB [movie_recommendation_system]> CREATE TABLE ratings (
    ->     rating_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     user_id INT,
    ->     movie_id INT,
    ->     rating INT CHECK (rating BETWEEN 1 AND 5),
    ->     FOREIGN KEY (user_id) REFERENCES users(user_id),
    ->     FOREIGN KEY (movie_id) REFERENCES movies(movie_id)
    -> );
Query OK, 0 rows affected (0.027 sec)

MariaDB [movie_recommendation_system]> show tables
    -> ;
+----------------------------------------+
| Tables_in_movie_recommendation_system |
+----------------------------------------+
| movies                                 |
| ratings                                |
| users                                  |
+----------------------------------------+
3 rows in set (0.001 sec)

MariaDB [movie_recommendation_system]> desc movie;
```

```
B [movie_recommendation_system]> INSERT INTO ratings (user_id, mo
ing) VALUES
 (1,1,5),(1,2,4),(2,1,4),(3,2,5);
OK, 4 rows affected (0.004 sec)
s: 4  Duplicates: 0  Warnings: 0
```

```
MariaDB [movie_recommendation_system]> desc movies;
+--------------+--------------+------+-----+---------+----------------+
| Field        | Type         | Null | Key | Default | Extra          |
+--------------+--------------+------+-----+---------+----------------+
| movie_id     | int(11)      | NO   | PRI | NULL    | auto_increment |
| title        | varchar(100) | YES  |     | NULL    |                |
| genre        | varchar(50)  | YES  |     | NULL    |                |
| release_year | int(11)      | YES  |     | NULL    |                |
+--------------+--------------+------+-----+---------+----------------+
4 rows in set (0.021 sec)
```

DML OPERATION

1. INSERT Operation

INSERT INTO movies (title, genre, release_year) VALUES ('Inception','Sci-Fi',2010), ('Interstellar','Sci-Fi',2014), ('Titanic','Romance',1997);

```
Query OK, 3 rows affected (0.006 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

INSERT INTO users (username) VALUES ('Amit'),('Rahul'),('Sujit');

```
MariaDB [movie_recommendation_system]> INSERT INTO users (username) VALUES
    -> ('Amit'),('Rahul'),('Sujit');
Query OK, 3 rows affected (0.002 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

INSERT INTO ratings (user_id, movie_id, rating) VALUES (1,1,5),(1,2,4),(2,1,4),(3,2,5);

```
MariaDB [movie_recommendation_system]> INSERT INTO ratings (user_id, movie_i
d, rating) VALUES
    -> (1,1,5),(1,2,4),(2,1,4),(3,2,5);
Query OK, 4 rows affected (0.004 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
MariaDB [movie_recommendation_system]> SELECT *FROM MOVIES;
+----------+-------------+---------+--------------+
| movie_id | title       | genre   | release_year |
+----------+-------------+---------+--------------+
|        1 | Inception   | Sci-Fi  |         2010 |
|        2 | Interstellar | Sci-Fi  |         2014 |
|        3 | Titanic     | Romance |         1997 |
+----------+-------------+---------+--------------+
3 rows in set (0.001 sec)

MariaDB [movie_recommendation_system]> SELECT * FROM users;
+---------+----------+
| user_id | username |
+---------+----------+
|       1 | Amit     |
|       2 | Rahul    |
|       3 | Sujit    |
+---------+----------+
3 rows in set (0.001 sec)

MariaDB [movie_recommendation_system]> SELECT * FROM ratings;
+-----------+---------+----------+--------+
| rating_id | user_id | movie_id | rating |
+-----------+---------+----------+--------+
|         1 |       1 |        1 |      5 |
|         2 |       1 |        2 |      5 |
|         4 |       3 |        2 |      5 |
+-----------+---------+----------+--------+
3 rows in set (0.000 sec)
```

2. UPDATE Operation

UPDATE ratings
SET rating = 5
WHERE user_id = 1 AND movie_id = 2;

```
MariaDB [movie_recommendation_system]> UPDATE ratings
    -> SET rating = 5
    -> WHERE user_id = 1 AND movie_id = 2;
Query OK, 1 row affected (0.005 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

3. DELETE Operation

DELETE FROM ratings
WHERE user_id = 2;

```
MariaDB [movie_recommendation_system]> DELETE FROM ratings
    -> WHERE user_id = 2;
Query OK, 1 row affected (0.019 sec)
```

Query 1: SELECT with Condition

SELECT title, release_year
FROM movies
WHERE genre = 'Sci-Fi';

```
MariaDB [movie_recommendation_system]> SELECT title, release_year
    -> FROM movies
    -> WHERE genre = 'Sci-Fi';
+--------------+--------------+
| title        | release_year |
+--------------+--------------+
| Inception    |         2010 |
| Interstellar |         2014 |
+--------------+--------------+
2 rows in set (0.001 sec)
```

Query 2: JOIN Query

SELECT u.username, m.title, r.rating
FROM users u
JOIN ratings r ON u.user_id = r.user_id
JOIN movies m ON r.movie_id = m.movie_id;

```
MariaDB [movie_recommendation_system]> SELECT u.username, m.title, r.rating
    -> FROM users u
    -> JOIN ratings r ON u.user_id = r.user_id
    -> JOIN movies m ON r.movie_id = m.movie_id;
+----------+--------------+--------+
| username | title        | rating |
+----------+--------------+--------+
| Amit     | Inception    |      5 |
| Amit     | Interstellar |      5 |
| Sujit    | Interstellar |      5 |
+----------+--------------+--------+
3 rows in set (0.001 sec)
```

Query 3: Aggregate Function

SELECT m.title, AVG(r.rating) AS average_rating
FROM movies m
JOIN ratings r ON m.movie_id = r.movie_id
GROUP BY m.movie_id;

```
MariaDB [movie_recommendation_system]> SELECT m.title, AVG(r.rating) AS average_rat
ing
    -> FROM movies m
    -> JOIN ratings r ON m.movie_id = r.movie_id
    -> GROUP BY m.movie_id;
+--------------+----------------+
| title        | average_rating |
+--------------+----------------+
| Inception    |         5.0000 |
| Interstellar |         5.0000 |
+--------------+----------------+
2 rows in set (0.002 sec)
```

Query 4: Subquery

```
SELECT title
FROM movies
WHERE movie_id NOT IN (
   SELECT movie_id
   FROM ratings
   WHERE user_id = 1
);
```

```
MariaDB [movie_recommendation_system]> SELECT title
    -> FROM movies
    -> WHERE movie_id NOT IN (
    ->     SELECT movie_id
    ->     FROM ratings
    ->     WHERE user_id = 1
    -> );
+---------+
| title   |
+---------+
| Titanic |
+---------+
1 row in set (0.002 sec)
```

Query 5: Recommendation Query

```
SELECT m.title, AVG(r.rating) AS avg_rating
FROM movies m
JOIN ratings r ON m.movie_id = r.movie_id
GROUP BY m.movie_id
ORDER BY avg_rating DESC
LIMIT 3;
```

```
MariaDB [movie_recommendation_system]> SELECT m.title, AVG(r.rating) AS avg_rating
    -> FROM movies m
    -> JOIN ratings r ON m.movie_id = r.movie_id
    -> GROUP BY m.movie_id
    -> ORDER BY avg_rating DESC
    -> LIMIT 3;
+--------------+------------+
| title        | avg_rating |
+--------------+------------+
| Interstellar |     5.0000 |
| Inception    |     5.0000 |
+--------------+------------+
2 rows in set (0.001 sec)
```

Conclusion

This project successfully demonstrates the implementation of a Movie Recommendation System using MySQL.

By using SQL queries such as joins, subqueries, aggregate functions, and DML operations, meaningful movie recommendations can be generated based on user preferences and ratings.